

From Text to Meaning: How AI Understands "I love AI"

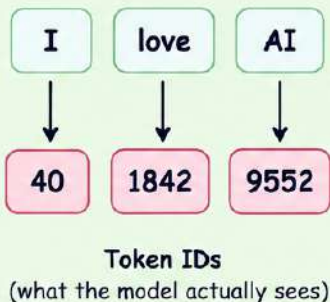
1 Raw text

"I love AI"

This is what we type in natural language (English).

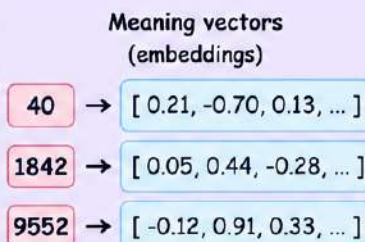
2 Tokenizer (problem 1)

The text is split into smaller pieces called tokens (words or subwords).



3 Embedding layer (problem 2)

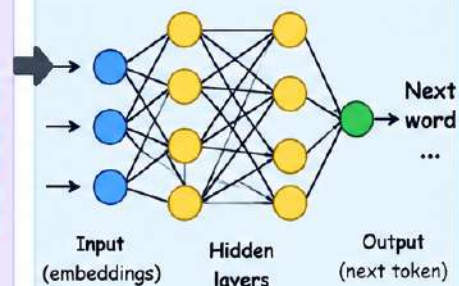
Each token ID is converted into a dense vector (embedding) that captures the meaning of the token.



Each vector can have hundreds or thousands of numbers (e.g., 768, 1024, 4096 dimensions).

4 Neural network can finally compute ✓

Now the model works with these numbers (vectors) to understand context, find patterns, and generate a response.



In short:

Text → Tokens → Token IDs → Meaning Vectors → Neural Network → Output

Byte Pair Encoding (BPE): How Subword Tokens are Learned

Start with characters, merge the most frequent pairs, and repeat!

BPE learns common subwords from data, so it can handle new words by breaking them into known pieces.

1 Start: every char is a token

Example word:

lowest

Initial vocabulary (only characters):

l o w e s t

At the beginning, each character is a token.

2 Count adjacent pairs

Look at all adjacent pairs in the word and count their frequency.

l o w e s t

Pair	Count
l o	1
o w	1
w e	1
e s	1
s t	1

Each pair appears once (because we have only one word here).

3 Most frequent pair

Pick the most frequent pair. Here: 'l' + 'o' → 'lo'

l o w e s t

lo

If there is a tie, we can pick any (often the one that appears first).

4 Re-count pairs

After merging, update the tokens and count pairs again.

lo w e s t

Pair	Count
lo w	1
w e	1
e s	1
s t	1

5 Next: merge again

Pick the next most frequent pair. Here: 'lo' + 'w' → 'low'

lo w e s t

low

Again, update the tokens and continue...

6 Repeat until vocab size reached

Keep merging the most frequent pairs from your training data (corpus). Over many words, you'll learn subwords like:

low est new er wid est play ing ...

We keep merging pairs (across all words in the corpus) until we reach the desired vocabulary size (e.g., 30,000 tokens).

7 Final vocabulary

The final vocabulary contains:

- All original characters
- Frequently merged subwords

Example final vocab (partial):

l o w e s t

lo low est new er ...

Result: Better tokenization

Word: lowest

Tokens (after BPE):

low est

Why BPE is useful?

- ✓ Handles new/unseen words (e.g., "newest" → "new" + "est")
- ✓ Smaller vocabulary than word-level
- ✓ Captures meaningful subwords
- ✓ Works well for many languages

 **Key Takeaway:** BPE starts with characters, repeatedly merges the most frequent adjacent pairs, and builds a vocabulary of characters + subwords. This helps the model handle rare and new words effectively.

Embeddings Capture Meaning: Similar Texts are Closer in Vector Space

Even if the words are different, similar meaning → similar vectors → higher cosine similarity

1 Convert text to embeddings

Each sentence is converted into a dense vector (e.g., using a model like text-embedding-3e5, BERT, etc.).

A "How to reset password"

[0.42, -0.18, **0.91**, 0.03, -0.27, ...]

0.91 is high → this dimension is strongly related to the meaning (e.g., account/security/reset)

B "Forgot my login"

[0.40, -0.12, **0.89**, 0.05, -0.30, ...]

0.89 is also high → similar meaning, so the vector looks similar to A

C "Pizza recipe"

[-0.31, 0.77, **-0.08**, 0.62, 0.14, ...]

-0.08 is low (or very different) → this dimension is **not** related to A

2 Measure similarity using Cosine Similarity

Cosine similarity measures the angle between two vectors. Closer vectors (smaller angle) = higher similarity.

$$\text{cosine}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

Value range:
-1 to 1
1 = very similar
0 = no similarity
-1 = opposite

Compare A and B (same meaning)

$\text{cosine}(A, B) = 0.82$ ✓ High similarity (Same meaning)

"How to reset password" and "Forgot my login" mean the same thing, so their vectors are close.

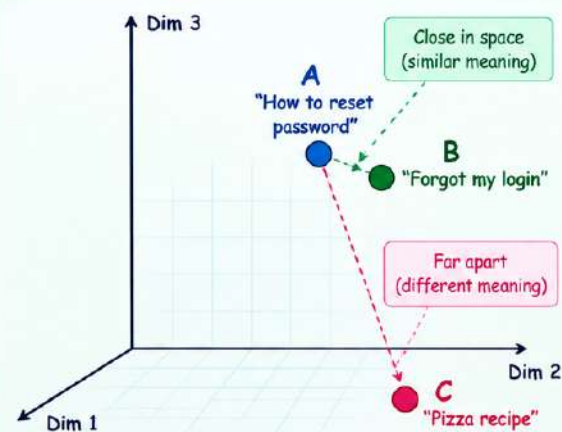
Compare A and C (different meaning)

$\text{cosine}(A, C) = 0.05$ ✗ Low similarity (Unrelated)

"How to reset password" and "Pizza recipe" have completely different meanings, so their vectors are far apart.

3 Visualization: Vectors in High-Dimensional Space

We can't show 1536 dimensions, but here's a simplified 3D view to illustrate the idea.



Cosine Similarity Interpretation

0.8 – 1.0	Very similar (same meaning)
0.3 – 0.8	Somewhat similar (related)
-0.3 – 0.3	No similarity
-1.0 – -0.3	Opposite meaning



Key Takeaway: Embeddings capture the meaning of text. Semantically similar sentences have similar vectors (higher cosine similarity), even if they use different words. This enables powerful use cases like search, RAG, clustering, and classification.

Cosine Similarity: Finding Similar Meaning in Vector Space

Different words, similar meaning → similar vectors → higher cosine similarity

1 Convert text to embeddings

Each sentence is converted into a dense vector using a model (e.g., text-embedding-3e5, BERT, etc.).

A "How to reset password"

[0.42, -0.18, **0.91**, 0.03, -0.27, ...]

0.91 is high → this dimension is strongly related to the meaning (e.g., account/security/reset)

B "Forgot my login"

[0.40, -0.12, **0.89**, 0.05, -0.30, ...]

0.89 is also high → similar meaning, so the vector looks similar to A

C "Pizza recipe"

[-0.31, 0.77, **-0.08**, 0.62, 0.14, ...]

-0.08 is low (or very different) → this dimension is **not** related to A

2 Cosine Similarity Formula

Cosine similarity measures the angle between two vectors. Closer vectors (smaller angle) = higher similarity.

$$\text{cosine}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

Where:

- $A \cdot B$ = dot product (sum of element-wise multiplication)
- $\|A\|$ = magnitude (length) of vector A
- $\|B\|$ = magnitude (length) of vector B
- Result range: -1 to 1
 - 1 = very similar (same direction)
 - 0 = no similarity (orthogonal)
 - -1 = opposite meaning

3 Calculate Similarity

A vs B $\text{cosine}(A, B) = 0.82$ ✓

High similarity (Same meaning)

"How to reset password" and "Forgot my login" mean the same thing, so their vectors are close.

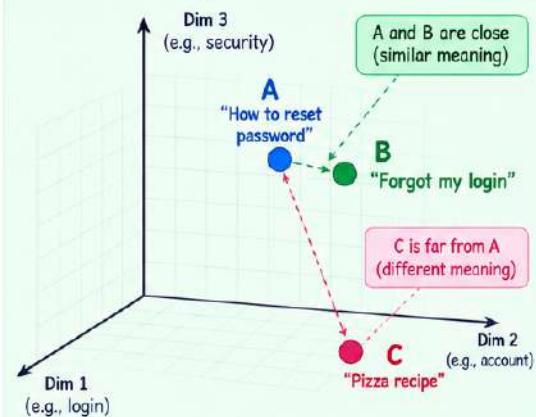
A vs C $\text{cosine}(A, C) = 0.05$ ✗

Low similarity (Unrelated)

"How to reset password" and "Pizza recipe" have completely different meanings, so their vectors are far apart.

4 Visualize in Vector Space (3D view)

Each sentence is a point in a high-dimensional space (here we show 3D for simplicity). Similar meanings are closer together.



Cosine Similarity Range (Interpretation)

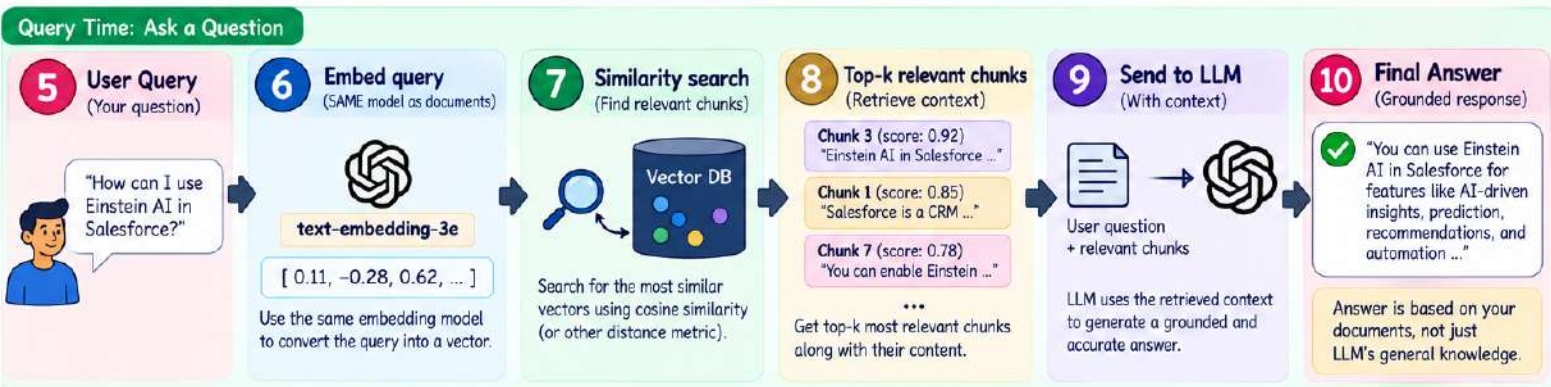
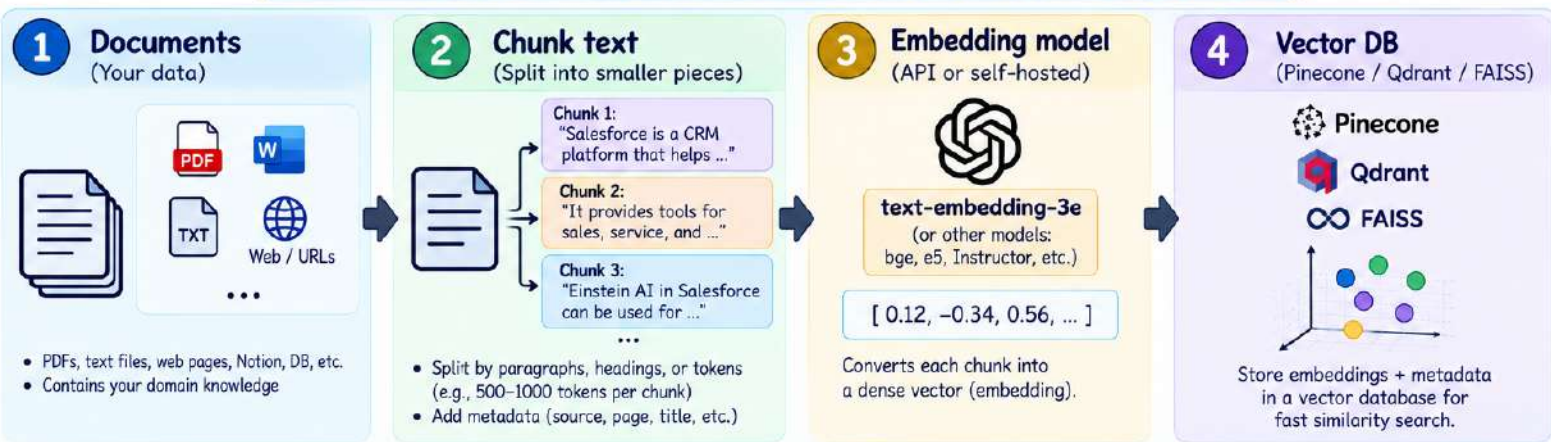
0.8 – 1.0	Very similar (same meaning)
0.3 – 0.8	Somewhat similar (related)
-0.3 – 0.3	No similarity
-1.0 – -0.3	Opposite meaning



Key Takeaway: Embeddings convert text into numeric vectors that capture **meaning**. Cosine similarity helps us find semantically similar text, even when different words are used. This is the foundation for search, RAG, clustering, classification, and more.

RAG Pipeline: From Documents to Answers

Store knowledge as vectors, find relevant chunks, and let LLM generate the answer



Key Takeaway: RAG combines the power of embeddings (for search) with LLMs (for generation) to give accurate, up-to-date, and domain-specific answers.

How Tokenization Actually Works

Example: "Tokenization is unbelievable"

Important:
Exact tokens and IDs depend on the tokenizer/model vocabulary.

1 Raw text (Input to tokenizer)

"Tokenization is unbelievable"



This is the original text we want to convert into tokens.

2 Normalize (Clean & standardize)

1. Lowercase

Convert text to lowercase (because most tokenizers are case-sensitive).

Tokenization → tokenization

2. Unicode normalization

Convert different unicode representations to a consistent form (e.g., NFC).

é → é (same character, consistent encoding)



This ensures the same text is always processed in a consistent way.

3 Pre-tokenize (Split into basic pieces)

Split the normalized text on spaces and punctuation (using rules like regex).

tokenization

is

unbelievable

These are initial pieces. They may still be further split using subword algorithms (like BPE).

4 Apply BPE merges (Build subword tokens)

Byte Pair Encoding (BPE) repeatedly merges frequent adjacent character or subword pairs based on a pre-trained vocabulary.

Example: "unbelievable"
(illustrative merges)

un + bel → unbel

unbel + ievable → unbelievable

The tokenizer has learned common subword merges from large text. Exact merges depend on the tokenizer's vocabulary.

5 Map tokens to IDs (Using vocabulary)

Each final token is looked up in the tokenizer's vocabulary and converted to a unique integer ID.

Token (subword)	Token ID
token	30001
ization	1634
is	318
un	403
believable	12373
	540

Final token IDs:

[30001, 1634, 318, 403, 12373, 540]



In short:

Raw text



Normalize



Pre-tokenize



BPE merges



Token IDs

Same text, same tokenizer
= same tokens and IDs!

Why Proper Tokenization & Embeddings are Important?

Let's see what happens if we take shortcuts...

1 Skip proper tokenization/ embeddings Use keyword matching only

Just check if the exact keyword is present in the text.



Problem:

❌ **'car' ≠ 'automobile'**
Query: car
Document: I love my automobile
Match: ❌ No match (even though they mean the same thing)

❌ **No meaning, no similarity**
Can't understand that "car", "auto" and "vehicle" mean the same thing.

❌ **Unknown words crash**
If a new word appears (not in the list), the system may fail or return nothing.

😞 **Exact word matching is too limited for real-world language!**

2 Use one-hot vectors Represent each word as a large sparse vector

Each word gets a vector with 1 at its index and 0 everywhere else (based on vocabulary size).

Example vocabulary (V = 6)

cat dog car bus train apple

One-hot vectors

cat	[0 0 0 0 0 0]
dog	[0 1 0 0 0 0]
car	[0 0 1 0 0 0]
bus	[0 0 0 1 0 0]
train	[0 0 0 0 1 0]
apple	[0 0 0 0 0 1]

Visual (cat vs car)

cat [1 0 0 0 0 0]

car [0 0 1 0 0 0]

They look completely different with no notion of similarity!

Problems:

❌ **Massive memory**
If we have 100,000 words, each vector has 100,000 dimensions!

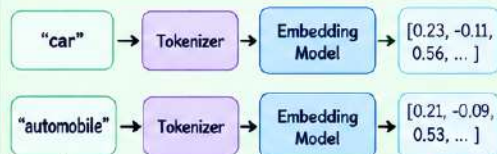
❌ **Every word equally distant**
In one-hot, all words are orthogonal (no notion of similarity).
Distance(cat, car) = Distance(cat, apple) = Distance(cat, train)

❌ **No generalization**
Can't capture meaning, relationships, or handle new/unseen words.

😞 **One-hot vectors are simple, but not practical for real-world AI!**

3 Use proper tokenization and embeddings Convert text to meaningful vectors

Break text into tokens, convert them into dense vectors (embeddings) that capture meaning, and let the model understand context and similarity.



Vectors are close in the embedding space (because they mean the same thing)

Benefits:

- ✅ **Understands meaning**
"car" and "automobile" have similar vectors.
- ✅ **Finds similarity**
Helps in semantic search, recommendations, etc.
- ✅ **Handles new words**
Even unseen words can be understood using subword tokenization (e.g., BPE, WordPiece, SentencePiece).
- ✅ **Powers real AI applications**
Used in RAG, chatbots, search, classification, clustering, and more.

😊 **Proper tokenization + embeddings = Meaning, similarity, and intelligence!**



Takeaway: Keyword matching and one-hot vectors are too naive. Proper tokenization and embeddings allow AI to understand language, not just match words.

From Text Input to Real AI Applications

How "I love AI" becomes meaning, and how it's used

1 Text Input

"I love AI"



This is the raw text we give to the model (e.g., a prompt, search query, document, etc.)

2 Tokenization

(Problem 1: How to split text?)

Text is split into smaller pieces called tokens using different strategies.

Word-level ['I', 'love', 'AI']

Character-level ['I', ' ', 'l', 'o', 'v', 'e', ' ', 'A', 'I']

Subword (most common)

BPE (GPT / tiktoken) ['I', 'love', 'AI']

WordPiece (BERT) ['I', 'love', 'AI']

SentencePiece ['_I', '_love', '_AI']

Subword tokenization (BPE, WordPiece, SentencePiece) is widely used as it handles new words and multiple languages well.

3 Token IDs

(Output of tokenizer)

Each token is mapped to a unique integer ID from the model's vocabulary.

Token	Token ID
I	40
love	1842
AI	9552

Token IDs
[40, 1842, 9552]
(what the model actually sees)

4 Embeddings

(Problem 2: Give meaning)

Each token ID is converted into a dense vector (embedding) that captures the meaning of the token.

Token ID	Embedding vector (only few values shown)
40	[0.21, -0.70, 0.13, ...]
1842	[0.05, 0.44, -0.28, ...]
9552	[-0.12, 0.91, 0.33, ...]

Embeddings
[[[0.21, -0.70, 0.13, ...],
[0.05, 0.44, -0.28, ...],
[-0.12, 0.91, 0.33, ...]]]
Each vector can have hundreds or thousands of dimensions (e.g., 768, 1024, 4096).

5 Contextual Understanding

(Transformer layers)

The embeddings go through multiple transformer layers (self-attention) to understand context, relationships, and generate a meaningful response.

Input embeddings (from previous step)

Transformer Layer 1

Transformer Layer 2

...

Transformer Layer N

Contextualized embeddings (sentence representation)

[0.02, -0.11, 0.56, ...]

6 Sentence Embeddings

(e.g., text-embedding-3e5 / bge)

For many applications, we take the final sentence/text embedding using models like:

- OpenAI text-embedding-3e5
- BGE (BAAI)
- Instructor, E5, etc.

Sentence embedding (for "I love AI")

[0.12, -0.34, 0.77, ...]

(e.g., 1536 dimensions)

7 Used by: Real AI Applications

These embeddings (token or sentence level) are used in many downstream tasks.



Similarity search

Find similar texts (e.g., search engines, recommendations)



RAG (Retrieval Augmented Generation)

Retrieve relevant documents and generate better responses



Clustering

Group similar texts (e.g., customer feedback, document clustering)



Classification

Classify text into categories (e.g., spam detection, sentiment analysis)



Text → Meaning
→ Real Impact!

Same embeddings can power chatbots, search, recommendation systems, analytics, and more!

From words to vectors, from vectors to intelligence.

AI understands language because we teach it to see meaning in numbers.

From Text to Meaning: How AI Understands "I love AI"

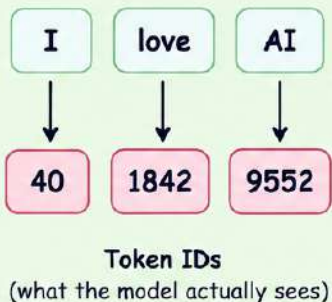
1 Raw text

"I love AI"

This is what we type in natural language (English).

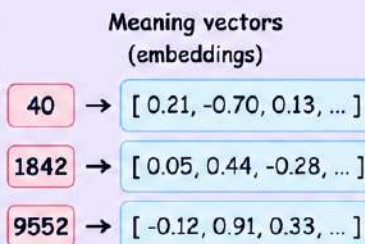
2 Tokenizer (problem 1)

The text is split into smaller pieces called tokens (words or subwords).



3 Embedding layer (problem 2)

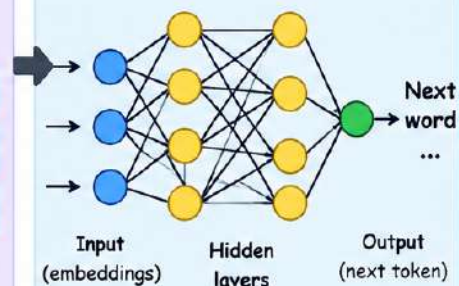
Each token ID is converted into a dense vector (embedding) that captures the meaning of the token.



Each vector can have hundreds or thousands of numbers (e.g., 768, 1024, 4096 dimensions).

4 Neural network can finally compute ✓

Now the model works with these numbers (vectors) to understand context, find patterns, and generate a response.



In short:

Text → Tokens → Token IDs → Meaning Vectors → Neural Network → Output